

Adding a Caching Service (Redis)

1. Overview

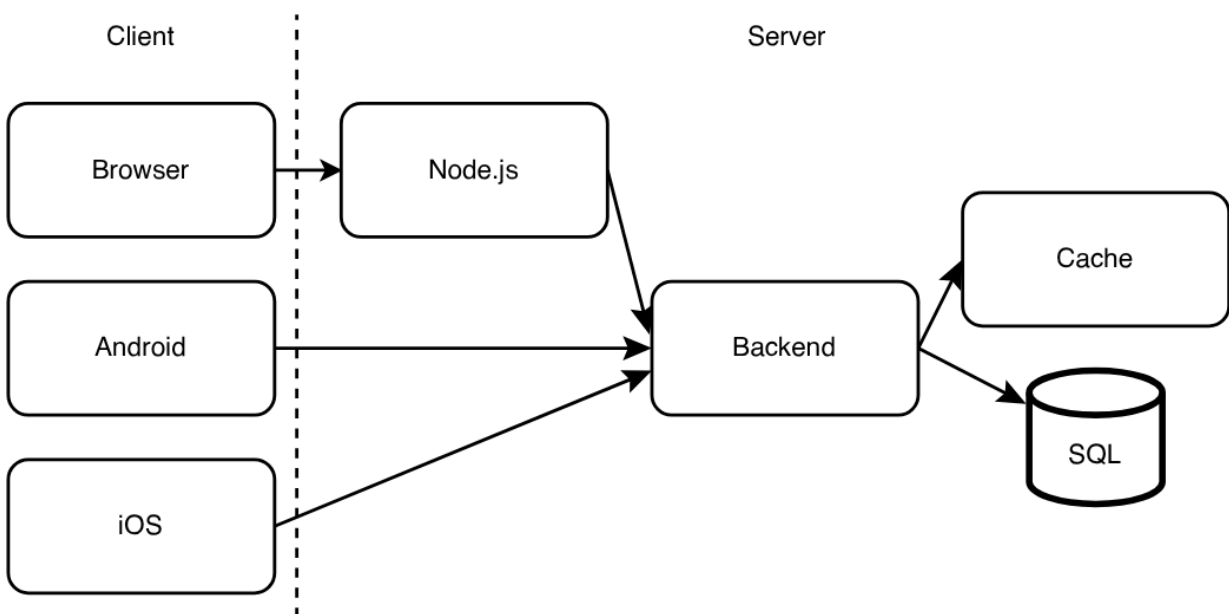
As traffic grows, the **database becomes a bottleneck**:

- High read load
- Increased latency
- Risk of failures

Solution:

Introduce a **caching layer (Redis)** to serve frequently accessed data

2. Updated Architecture



3. Request Flow (Cache-Aside Pattern)

Step-by-Step:

1. Request comes to backend
2. Backend checks **Redis cache**

Case 1: Cache Hit

- Data found in cache
- Return immediately

Fast response (milliseconds)

Case 2: Cache Miss

- Fetch from database
- Store in cache
- Return response

4. What to Cache

Not everything should be cached.

Good Candidates:

- Frequently accessed data
- Read-heavy endpoints
- Rarely changing data

Examples:

- User profiles
- Feed data
- Popular posts

Avoid Caching:

- Highly dynamic data
- Sensitive data
- Write-heavy data

5. Why Redis?

- In-memory (very fast)
- Supports key-value storage
- Supports TTL (auto-expiry)
- Scales easily

6. Benefits of Caching

1. Reduced Database Load

- Fewer queries

2. Lower Latency

- Faster response

3. Improved Throughput

- Handle more requests

4. Better User Experience

7. Cache Invalidation

“There are only two hard things in CS: cache invalidation and naming things.”

Problem:

- Data in cache becomes stale

Solutions:

1. TTL (Time-To-Live)

- Auto expire after time

2. Write-Through

- Update cache + DB together

3. Cache Eviction on Update

- Delete cache when DB updates

8. Caching Strategies

1. Cache Aside (Lazy Loading)

- Load only when needed

2. Write Through

- Write to cache + DB simultaneously

3. Write Back

- Write to cache first, DB later

9. Challenges

1. Cache Miss Storm

- Many requests hit DB simultaneously

Solution:

- Request coalescing
- Locking

2. Stale Data

- Inconsistent responses

3. Cache Eviction

- Limited memory

4. Cold Start Problem

- Empty cache → high DB load

10. Advanced Concepts

Distributed Cache

- Multiple Redis nodes

Cache Partitioning

- Use consistent hashing

CDN (Edge Caching)

- Cache at global level

11. Evolution from Previous System

Before:

- Backend → DB directly

Now:

- Backend → Cache → DB

Reduces DB pressure significantly

12. Real-World Usage

- Instagram → feed caching
- YouTube → video metadata
- Amazon → product pages

13. Final Summary

- Cache sits between backend and DB
- Reduces DB load and latency
- Uses **cache-aside pattern**
- Requires careful **cache invalidation**
- Redis is commonly used